SERVICIO NACIONAL DE APRENDIZAJE – SENA (National Learning Service)

CENTER FOR INDUSTRY, BUSINESS AND SERVICES (CIES)

HUILA REGIONAL

Training Program: Technology in Software Analysis and Development (ADSO)

Group (Ficha): 3145556

INSTALLATION MANUAL

Project: Electrical Consumption Monitor

Procedure for deploying the API gateway and the services, the message queue, the MQTT broker, the measuring device, the web application and the mobile application

Item	Information
Team members	Juan Esteban Cortés Parra, Miguel Ángel García Artunduaga
Instructor	Instructor team of ficha 3145556
Training center	Center for Industry, Business and Services (CIES) – SENA Huila Regional
Program	Technology in Software Analysis and Development (ADSO)
Group (Ficha)	3145556
Document type	Installation manual
Version	1.4
Date prepared	October 7, 2026 (update: October 28, 2026)

Year 2026

2. Document version control

The change history of this manual is recorded in the following table. Every modification to the installation process that affects the software must be reflected in a new version of the document.

Version	Date	Description of change	Responsible
1.0	10/07/2026	Initial creation of the installation manual for the backend, web frontend and mobile frontend, with requirements from the requirements specification and references in APA format.	Development team
1.1	10/13/2026	Update to the final state: five system parts, complete environment variables, MQTT broker, firmware, linking the first device, production, mobile application integrated with the API, and expanded verifications and troubleshooting.	Development team
1.2	10/24/2026	Services architecture: cloning of all repositories (including `en-monitor-measurement-service`), infrastructure in `energy-monitor-infra` (full stack and development mode with MySQL, RabbitMQ and Mosquitto), ports 8080, 8082, 8083, 8084 and 8085, per-service variables (`MQTT_DEVICE_SERVICE_*`, `MQTT_MEASUREMENT_SERVICE_*` instead of `MQTT_BACKEND_*`, `INTERNAL_API_TOKEN`, `RABBITMQ_*`, `MEASUREMENT_CONTEXT`), broker authenticated by the device service, firmware in `energy-monitor-backend/firmware/`, updated verifications and troubleshooting.	Development team
1.3	10/27/2026	Migration finished: seven microservices (security 8082, device 8084, measurement 8085, home 8086, notification 8087, alert 8088 and recommendation 8089) behind the gateway and backend retired; cloning, `.env` per service, start-up order, consumer and relay switches, first administrator with `make-admin.sh`, new end-to-end tests, updated checks and solutions.	Development team
1.4	10/28/2026	Unusual consumption alert in the alert service (no new variables or installation steps), profile and member photos, and the Settings help center; the final version is unified at 1.4.	Development team

3. Table of contents

2. Document version control

3. Table of contents

4. Introduction

5. Objective

6. Technical scope

7. Minimum requirements

- 7.1 Equipment requirements (according to the requirements specification)
- 7.2 Recommended hardware resources to run the system
- 7.3 Software requirements for the installation

8. Environment variables the installer must know

9. Installation of the backend services

- 9.1 Obtain the code · 9.2 Define the environment variables (.env) · 9.3 Start the infrastructure with Docker Compose · 9.4 Create the local files of each service · 9.5 Generate the RSA key pair (JWT tokens) · 9.6 Generate the recovery pepper · 9.7 Decide on the recovery email · 9.8 Build and start

10. Web frontend installation

- 10.1 Dependencies · 10.2 Configure the development gateway · 10.3 Start in development · 10.4 CORS permission in the gateway

11. Mobile application installation

- 11.1 Dependencies · 11.2 Configure the API URL · 11.3 Start Expo · 11.4 Test on a physical device · 11.5 Compatibility notes

12. MQTT broker

13. Measuring device firmware

14. Link the first device

15. Production

16. Local email with Mailpit (optional)

17. Installation verification

18. Installation command summary

19. Common troubleshooting

20. References (APA format, 7th edition)

4. Introduction

This document is the installation manual of the "Electrical Consumption Monitor" software. Its purpose is for any developer, instructor or technician —regardless of prior experience with the project— to be able to obtain the source code, configure the dependencies, start the components of the system and verify that they work together on a workstation.

The system is made up of these parts, each in its own repository: an API gateway, the single entry point of the API (Spring Cloud Gateway on Spring Boot 4.0.8), seven microservices —security, devices, measurements, homes, notifications, alerts and recommendations—, all built with Java 21 and Spring Boot 4.1.1; a web application built with React 19 and Vite 8; an Android mobile application built with Expo SDK 57 and React Native 0.86; an Eclipse Mosquitto MQTT broker that authenticates each client against the device service; and a measuring device made of an ESP32 board and a PZEM-004T V3 sensor (Spring, n.d.; Vite, n.d.; Expo, n.d.; Eclipse Foundation, n.d.). The former modular backend (`energy-monitor-backend`) is retired and is no longer installed: it only keeps the device firmware. The MySQL 8.4 database, the RabbitMQ 4.1 message queue and the broker are started with

Docker Compose from the infrastructure repository, and email delivery can be simulated locally with Mailpit (Docker Inc., n.d.; RabbitMQ, n.d.; Mailpit, n.d.).

The information recorded was obtained directly from the project's repositories (gateway, services, web, mobile and infrastructure), from the requirements specification and from the READMEs included in each repository, so it corresponds to the `dev` branch of the repositories on October 27, 2026. References were formatted according to APA style, seventh edition (American Psychological Association [APA], 2020).

5. Objective

To allow a new or maintenance development team to start the "Electrical Consumption Monitor" system on a Windows, macOS or Linux computer, from the official repositories, without depending on informal documentation or past conversations.

- List the minimum hardware, operating system and software requirements, consistent with the requirements specification.
- Establish exact versions of the tools the installation process requires (Java, Node, Docker, OpenSSL, Arduino).
- Describe, step by step and with the concrete commands, the installation of the database, the message queue, the backend services, the web application, the mobile application, the MQTT broker and the device firmware.
- Present the configuration variables the installer must define before the first start.
- Explain how to link the first device, how to deploy to production, how to verify that each component is operational and how to diagnose the most common errors.

6. Technical scope

The manual covers installation on a local development workstation of the following components and, in section 15, the summary of deployment on a production server with Docker Compose:

- MySQL 8.4, RabbitMQ 4.1 and the Mosquitto MQTT broker through Docker Compose (energy-monitor-infra), in two modes: the full stack in containers, or only the infrastructure with the services running on the machine (development mode).
- API gateway (en-monitor-api-gateway, port 8080).
- Security (en-monitor-security-service, 8082), device (en-monitor-device-service, 8084), measurement (en-monitor-measurement-service, 8085), home (en-monitor-home-service, 8086), notification (en-monitor-notification-service, 8087), alert (en-monitor-alert-service, 8088) and recommendation (en-monitor-recommendation-service, 8089) services, with the local profile and JUnit tests.
- Device firmware, in the `firmware/` folder of the retired `energy-monitor-backend` repository (cloned only for the firmware; the backend no longer runs).
- Web application (Energy-Monitor-application) with Vite.
- Android mobile application (Energy-Monitor-Mobile) with Expo and Expo Go.
- Utilities: `make-admin.sh` for the first administrator; optional, Mailpit for local email, your own RSA keys for the JWTs and the end-to-end tests in `energy-monitor-infra/e2e/`.

The `energy-monitor-devices` repository is retired: the firmware moved to `energy-monitor-backend/firmware/` and the broker configuration to `energy-monitor-infra/mosquitto/`. It does not cover hiring a server or a domain, nor the detailed configuration of a reverse proxy; the full production procedure is in the implementation and deployment manual.

7. Minimum requirements

7.1 Equipment requirements (according to the requirements specification)

According to the compatibility requirements of the requirements specification (RNF-COM), the software runs on computers and mobile devices in perfect condition, with an internet connection, enough storage to hold the application and the latest available version of the operating system (Cortés Parra & García Artunduaga, 2026):

- Computer operating system: Windows 10 or higher.
- Mobile operating system: the latest available version of Android. The mobile application is not available for iPhone.
- Up-to-date web browser: Chrome, Firefox, Safari or Edge. Linking the device over Bluetooth is only available in desktop Chrome or Edge and in Chrome for Android, and requires a secure connection (HTTPS or `localhost`).
- Internet connection (services communicate through standard HTTP/HTTPS protocols).
- For the measuring device: an ESP32 DevKit board with Wi-Fi and Bluetooth, a PZEM-004T V3 sensor, an HLK-PM03 power supply and a USB cable.

7.2 Recommended hardware resources to run the system

Beyond the equipment requirements stated in the requirements specification, the machine where the components are installed must have the following minimum resources for a smooth experience without freezes or memory errors. The exact software versions are detailed in the table of section 7.3.

Resource	Minimum	Recommended
Operating system (PC)	64-bit Windows 10 (with Docker Desktop and WSL 2)	64-bit Windows 11 or Linux (Ubuntu 24.04+)
Operating system (mobile)	Android 10 or higher	Latest available Android version
Processor	Intel Core i3 or equivalent	Intel Core i5 or equivalent
RAM	8 GB	16 GB (Docker Desktop, MySQL, RabbitMQ, the gateway and the seven Java services, Node and design tools open comfortably)
Free disk space	8 GB (code, node_modules, MySQL image and containers)	20 GB or more (for several versions, Docker caches and database files)
Connectivity	Stable internet (to clone, download packages and Expo Go)	Broadband connection
Additional storage	At least 5 GB free for test data and MySQL backups	Free space at your disposal

> *If the machine meets the minimum but not the recommendation, install only the components you need at the same time (e.g. use the full stack in containers instead of opening the eight Java services in the IDE, and do not open Android Studio at the same time).*

7.3 Software requirements for the installation

The minimum versions verified on the services repositories, the two frontends and the infrastructure repository are:

Tool	Minimum version	What it is used for
Java JDK	21 or higher	Build and run the gateway and the services (Spring Boot 4.1.1; the gateway, 4.0.8).
Node.js	22 LTS (min. 20.19)	Tooling for React 19/Vite 8 and Expo SDK 57.
npm	included with Node	Dependency management for web and mobile.
Docker Desktop / Docker Engine	24 or higher	Start MySQL 8.4, RabbitMQ 4.1, the Mosquitto broker, the full stack and Mailpit.
Docker Compose	2.24 or higher	Create the database schema and user and start the infrastructure; the development mode uses the `!override` tag.
OpenSSL and bash	any recent version	Generate the RSA key pair, the pepper, the secrets and the development certificates (`prepare.sh`).
Git	latest stable	Clone the repositories; on Windows it bundles OpenSSL.
Expo Go	store app	Test the mobile app without an emulator on a physical device.
arduino-cli or Arduino IDE 2	latest stable	Build and upload the device firmware (Espressif esp32 core 3.x).

All these tools are distributed with official online documentation (Oracle, n.d.; Docker Inc., n.d.; Node.js Foundation, n.d.; Git, n.d.; OpenSSL, n.d.; Expo, n.d.; Arduino, n.d.).

8. Environment variables the installer must know

No service stores credentials in the code: everything is injected through environment variables or Git-ignored files (`.env`, `application-local.yaml`). There are two configuration levels:

- `infra/.env` (copy of `energy-monitor-infra/.env.example`): read by Docker Compose; it serves the full stack and the infrastructure of the development mode.
- **Each service's** `.env` (copy of its `.env.example`) and its `application-local.yaml`: only in development mode, when the service runs on the machine. Shared values must be the same as in `infra/.env`; `MYSQL_PASSWORD` is called `SPRING_DATASOURCE_PASSWORD` there.

The names below are the exact ones in the `.env.example` and `application.yaml` files of each repository (Spring, n.d.).

> *The `.env` files are read as `.properties`: write values without quotes, without trailing spaces and without `$`. A value containing `$` is treated as a placeholder, and Compose interprets it too. Do not leave the `CHANGE_ME_...` example texts.*

Variable / file	What it is for	Where to set it
MYSQL_PASSWORD / MYSQL_ROOT_PASSWORD	Passwords of the services' `energy_app` user and of `root`. MySQL takes them only when its volume is created.	`infra/.env`
SPRING_DATASOURCE_PASSWORD	Same password as MYSQL_PASSWORD; used by each service in development mode and by its `test` profile tests.	`.env` of each service with a database
SECURITY_RESET_PEPPER	Long, random pepper (HMAC) for the recovery code.	`infra/.env` and the security service's `.env`
RABBITMQ_USERNAME / RABBITMQ_PASSWORD	Message queue user (`energy-monitor` by default). RabbitMQ creates it only when its volume is first started.	`infra/.env` and `.env` of each service

Variable / file	What it is for	Where to set it
INTERNAL_API_TOKEN	Token of the `/internal/**` calls between services; the same value in all of them.	`infra/.env` and `.env` of each service
SECURITY_CORS_ALLOWED_ORIGINS	Origins allowed for the browser; CORS is answered by the gateway (`http://localhost` in the full stack, `http://localhost:5173` in development mode).	`infra/.env` and the gateway's `.env`
SECURITY_JWT_PRIVATE_KEY_PATH / SECURITY_JWT_PUBLIC_KEY_PATH	Absolute paths to the PEM keys of the JWT token. Only the security service uses the private key; the gateway and the other services use the public one.	Each service's `.env` (development mode)
SECURITY_SESSION_COOKIE_SECURE	`false` in development mode: the web session travels in HttpOnly cookies and the local machine is served over `http://`.	The security service's `.env`
GOOGLE_CLIENT_ID / GOOGLE_CLIENT_SECRET	OAuth credentials ("Web application") to sign in with Google. If empty, the Google button does not appear.	`infra/.env` and the security service's `.env`
MAIL_HOST, MAIL_PORT, MAIL_USERNAME, MAIL_PASSWORD, MAIL_FROM	SMTP for the verification, recovery and deletion codes and the alert, recommendation and activity emails. Only the notification service sends email.	`infra/.env` and the notification service's `.env`
MQTT_BROKER_URL	Address of the broker's internal listener (`tcp://localhost:1883` in development mode).	`.env` of devices and measurements
MQTT_DEVICE_SERVICE_USER / MQTT_DEVICE_SERVICE_PASS	Credentials of the device service in the broker (`em-device-service`). Random password (`openssl rand -hex 24`).	`infra/.env` and the device service's `.env`
MQTT_MEASUREMENT_SERVICE_USER / MQTT_MEASUREMENT_SERVICE_PASS	Credentials of the measurement service in the broker (`em-measurement-service`). The device service checks them, so they must match in both.	`infra/.env` and `.env` of measurements and devices
MEASUREMENT_MQTT_ENABLED	`true` so the measurement service subscribes to telemetry in development mode.	The measurement service's `.env`
MQTT_CLIENT_ID	Identifier in the broker; it must differ per instance of the device or measurement services.	Environment variable (optional)
MQTT_AUTH_ALLOWED_NETWORKS	Networks allowed to call the device service's `/internal/mqtt/auth/**` (default `127.0.0.1/32,::1/128,172.16.0.0/12`).	Environment variable (optional)
DEVICE_MQTT_PUBLIC_HOST / DEVICE_MQTT_PUBLIC_PORT	Broker address the web writes to the device when linking it. Never `localhost`: in development, the LAN IP of the PC; in production, the broker's domain. Port `8883`.	`infra/.env` and the device service's `.env`
DEVICE_OFFLINE_AFTER	Silence time after which a device is marked offline (default `PT150S`); used by the device service.	Environment variable (optional)

Variable / file	What it is for	Where to set it
SECURITY_SERVICE_URL, DEVICE_SERVICE_URL, MEASUREMENT_SERVICE_URL, HOME_SERVICE_URL, NOTIFICATION_SERVICE_URL, ALERT_SERVICE_URL, RECOMMENDATION_SERVICE_URL	Addresses between services in development mode (`http://localhost:8082`, `:8084`, `:8085`, `:8086`, `:8087`, `:8088` and `:8089`). The gateway uses all of them; each service, those it calls. The `.env.example` files already carry the local values.	Each service's `.env`
NOTIFICATION_WEB_URL	Address of the web used by the links in the emails (`http://localhost` in the full stack, `http://localhost:5173` in development mode).	`infra/.env` and the notification service's `.env`
NOTIFICATION_VAPID_PUBLIC_KEY / NOTIFICATION_VAPID_PRIVATE_KEY	VAPID key pair for browser push notifications. Without it push is disabled and email keeps working. They must always be the same in an environment.	`infra/.env` and the notification service's `.env`
NOTIFICATION_VAPID_SUBJECT	Contact associated with the VAPID keys (a support `mailto:` by default).	Environment variable (optional)
EXPO_ACCESS_TOKEN	expo.dev token for mobile app push. Goes on the server only, never in the app.	`infra/.env` and the notification service's `.env`
ALERT_LIMIT_ZONE	Time zone in which the day and month of limit alerts start (default `America/Bogota`).	Alert service environment variable (optional)
RECOMMENDATION_EVALUATE_INTERVAL / RECOMMENDATION_ZONE / RECOMMENDATION_INITIAL_DELAY	How often the saving rules run (`PT1H`), the time zone of the day-part rules (`America/Bogota`) and the wait before the first evaluation (`PT2M`).	Recommendation service environment variables (optional)
SECURITY_EVENTS_CONSUMER_ENABLED, ALERT_EVENTS_CONSUMER_ENABLED, ACTIVITY_CONSUMER_ENABLED, HOME_OUTBOX_RELAY_ENABLED, ACCOUNT_DELETED_CONSUMER_ENABLED, ALERT_CONSUMERS_ENABLED, ALERT_SERVICE_OUTBOX_RELAY_ENABLED, RECOMMENDATION_SCHEDULER_ENABLED, RECOMMENDATION_OUTBOX_RELAY_ENABLED	Switches of the consumers and relays of the notification, home, alert and recommendation services. All `true` by default; `false` only served to coexist with the retired backend.	`infra/.env` (optional; do not set them)
SECURITY_ACCOUNT_DELETION_CODE_MAIL_ENABLED	Mailing of the deletion code to Google accounts (`false` by default). Turn it on only with the notification service deployed.	`infra/.env` and the security service's `.env` (optional)
SECURITY_CONTEXT, DEVICE_CONTEXT, MEASUREMENT_CONTEXT, HOME_CONTEXT, NOTIFICATION_CONTEXT, ALERT_CONTEXT, RECOMMENDATION_CONTEXT, GATEWAY_CONTEXT, FRONTEND_CONTEXT	Paths of the clones; only if the repositories are not side by side (section 9.1).	`infra/.env`
VITE_DEV_API_TARGET	Gateway to which the Vite server forwards `/api` (default `http://localhost:8080`).	`.env.local` in Energy-Monitor-application (optional)

Variable / file	What it is for	Where to set it
VITE_GOOGLE_CLIENT_ID	The same client ID as GOOGLE_CLIENT_ID. Empty: the Google button is not shown.	`infra/.env` (full stack) and `.env.local` in Energy-Monitor-application
EXPO_PUBLIC_API_BASE_URL	API URL for the mobile app, without `/api/v1` (section 11.2).	`.env` in Energy-Monitor-Mobile
EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID	Google **web** client ID (the same as GOOGLE_CLIENT_ID) for the app's native sign-in.	`.env` in Energy-Monitor-Mobile (optional)
EXPO_PUBLIC_EAS_PROJECT_ID	Expo project identifier; without it the app does not register the phone to receive push.	`.env` in Energy-Monitor-Mobile

> *Variables retired with the backend:* `BACKEND_URL`, `BACKEND_INTERNAL_URL`, `BACKEND_CONTEXT`, `MQTT_BACKEND_USER` *and* `MQTT_BACKEND_PASS`. *If they are still in an old* `.env`, *they are unused and can be deleted.*

9. Installation of the backend services

Repositories: `en-monitor-api-gateway` (port 8080), `en-monitor-security-service` (8082), `en-monitor-device-service` (8084), `en-monitor-measurement-service` (8085), `en-monitor-home-service` (8086), `en-monitor-notification-service` (8087), `en-monitor-alert-service` (8088) and `en-monitor-recommendation-service` (8089), all with Java 21 and Spring Boot (4.1.1; the gateway, 4.0.8), MySQL 8.4, Liquibase and JWT RS256, plus `energy-monitor-infra` with Docker Compose. The API is always consumed through the gateway, at `http://localhost:8080` in development mode or `http://localhost` with the full stack (Spring, n.d.).

9.1 Obtain the code

1. Clone the repositories side by side, with this folder structure (infra builds them from there):

```
mkdir energy-monitor && cd energy-monitor
git clone https://github.com/Juan-Cortes2045/energy-monitor-infra.git infra
# Retired: cloned only because it keeps the firmware (prepare.sh writes dev_ca.h into it).
git clone https://github.com/Juan-Cortes2045/energy-monitor-backend.git backend/energy-monitor-backend
git clone https://github.com/Juan-Cortes2045/en-monitor-security-service.git backend/en-monitor-security-service
git clone https://github.com/Juan-Cortes2045/en-monitor-device-service.git backend/en-monitor-device-service
git clone https://github.com/Juan-Cortes2045/en-monitor-measurement-service.git backend/en-monitor-measurement-service
git clone https://github.com/angelgarcia0/en-monitor-home-service.git backend/en-monitor-home-service
git clone https://github.com/angelgarcia0/en-monitor-notification-service.git backend/en-monitor-notification-service
git clone https://github.com/angelgarcia0/en-monitor-alert-service.git backend/en-monitor-alert-service
git clone https://github.com/angelgarcia0/en-monitor-recommendation-service.git backend/en-monitor-recommendation-service
git clone https://github.com/Juan-Cortes2045/en-monitor-api-gateway.git backend/en-monitor-api-gateway
git clone https://github.com/Juan-Cortes2045/Energy-Monitor-application.git frontend/energy-monitor-web
git clone https://github.com/angelgarcia0/Energy-Monitor-Mobile.git frontend/energy-monitor-mobile
```

2. Verify that each service contains `mvnw`, `pom.xml`, `Dockerfile` and the `src/` folder, and that `infra/` contains `docker-compose.yml`, `docker-compose.dev.yml` and `prepare.sh`. If a repository ends up in another path, set it in `infra/.env` with its `*_CONTEXT` variable (section 8).

9.2 Define the environment variables (.env)

Copy the infrastructure example file and protect it. The `.env` file is in `.gitignore` and must not be uploaded to the repository:

```
cd infra
cp .env.example .env && chmod 600 .env
# On Windows PowerShell:
Copy-Item .env.example .env
```

Generate the random secrets that are empty (the block prints none of them):

```
gen() { grep -q "^$1=$" .env && sed -i "s|^$1=$|$1=$(openssl rand -hex "$2")|" .env; }
gen MYSQL_PASSWORD 16;    gen MYSQL_ROOT_PASSWORD 16; gen SECURITY_RESET_PEPPER 32
gen RABBITMQ_PASSWORD 24; gen INTERNAL_API_TOKEN 32;  gen MQTT_MEASUREMENT_SERVICE_PASS 24
gen MQTT_DEVICE_SERVICE_PASS 24
```

Set the machine's LAN IP in `DEVICE_MQTT_PUBLIC_HOST` (on Linux, `hostname -I`) and fill in, if used, email, Google, VAPID and Expo (section 8). Check that no required variable is missing and no example value is left:

```
docker compose config -q && echo "compose OK"         # if a required one is missing, it names it
grep -lE '^[A-Z_]+=.*CHANGE_ME' .env ../backend/*/.env  # must not list any file
```

9.3 Start the infrastructure with Docker Compose

First, from `infra/`, generate the JWT RSA pair (`infra/keys/`) and the broker's development certificates. Repeat it whenever the LAN IP changes:

```
./prepare.sh
```

Then choose a mode:

- **Full stack (everything in containers).** It is the fastest way to have the system running; sections 9.4 to 9.8 are not needed:

```
docker compose up -d --build
docker compose ps        # gateway, security, device, measurement, home, notification, alert and
recommendation must end up (healthy)
```

The web is at `http://localhost` and the API at `http://localhost/api/v1/...`, through the gateway. The services do not publish ports in this mode.

- **Development mode (services on the machine).** Start only MySQL, RabbitMQ and the broker, and then each service from the IDE or the terminal (sections 9.4 to 9.8):

```
docker compose stop gateway security device measurement home notification alert recommendation
frontend   # if the full stack was up
docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d mysql rabbitmq mosquitto
docker compose ps
```

Wait for `energy-monitor-mysql` and `energy-monitor-rabbitmq` to report "healthy". Docker creates the `energy_monitor` schema and the `energy_app` user with the values from `.env`; both modes use the same database, the same queue and the same broker (Docker Inc., n.d.). The RabbitMQ console is at `http://localhost:15672`.

9.4 Create the local files of each service

Development mode only. In each service, copy its `.env.example` as `.env` and, if present, its `src/main/resources/application-local.yaml.example` as `application-local.yaml`:

```
cd ../backend/en-monitor-security-service
cp .env.example .env
cp src/main/resources/application-local.yaml.example src/main/resources/application-local.yaml
# Same in the device, measurement, home, notification, alert and recommendation services.
# en-monitor-api-gateway only has .env.example.
```

Fill in each `.env` with the values from `infra/.env`:

File	Values from `infra/.env`	Specific to development mode
`backend/en-monitor-security-service/.env`	`MYSQL_PASSWORD` (as `SPRING_DATASOURCE_PASSWORD`), `SECURITY_RESET_PEPPER`, `GOOGLE_CLIENT_ID/SECRET`, `RABBITMQ_*`, `INTERNAL_API_TOKEN` (and `HOME_SERVICE_URL=http://localhost:8086`)	`SECURITY_JWT_PRIVATE_KEY_PATH`, `SECURITY_JWT_PUBLIC_KEY_PATH`, `SECURITY_SESSION_COOKIE_SECURE=false`
`backend/en-monitor-device-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`, `MQTT_MEASUREMENT_SERVICE_USER/PASS`, `MQTT_DEVICE_SERVICE_USER/PASS`, `DEVICE_MQTT_PUBLIC_HOST`	`SECURITY_JWT_PUBLIC_KEY_PATH`
`backend/en-monitor-measurement-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`, `MQTT_MEASUREMENT_SERVICE_USER/PASS`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `MEASUREMENT_MQTT_ENABLED=true`
`backend/en-monitor-home-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `SECURITY_SERVICE_URL=http://localhost:8082`
`backend/en-monitor-notification-service/.env`	`MYSQL_PASSWORD`, `MAIL_*`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`, VAPID, `EXPO_ACCESS_TOKEN`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `NOTIFICATION_WEB_URL=http://localhost:5173`, `SECURITY_SERVICE_URL`, `HOME_SERVICE_URL`, `DEVICE_SERVICE_URL` (on `localhost`)
`backend/en-monitor-alert-service/.env` and `backend/en-monitor-recommendation-service/.env`	`MYSQL_PASSWORD`, `RABBITMQ_*`, `INTERNAL_API_TOKEN`	`SECURITY_JWT_PUBLIC_KEY_PATH`, `HOME_SERVICE_URL`, `DEVICE_SERVICE_URL`, `MEASUREMENT_SERVICE_URL` (on `localhost`)
`backend/en-monitor-api-gateway/.env`	—	`SECURITY_JWT_PUBLIC_KEY_PATH`, `SECURITY_CORS_ALLOWED_ORIGINS=http://localhost:5173`

In the `application-local.yaml` files, the `spring.datasource.password` value must be the same as `MYSQL_PASSWORD`. To check that a shared value is the same in every file without showing it, compare their digests from `infra/`:

```
for v in RABBITMQ_PASSWORD INTERNAL_API_TOKEN MQTT_MEASUREMENT_SERVICE_PASS; do
  for f in .env ../backend/*/.env; do
    grep -q "^$v=" "$f" && echo "$v $(grep "^$v=" "$f" | cut -d= -f2- | sha256sum | cut -c1-8)
$f"
  done
done      # each variable must give the same digest in all its files
```

9.5 Generate the RSA key pair (JWT tokens)

Access tokens are signed with RS256. In the full stack the pair is generated by `prepare.sh` in `infra/keys/`, and the development-mode services can point to those same keys. They can also use their own pair. On Linux and macOS:

```
mkdir -p ~/.energy-monitor/keys
chmod 700 ~/.energy-monitor/keys
openssl genpkey -algorithm RSA -out ~/.energy-monitor/keys/private.pem -pkeyopt
rsa_keygen_bits:2048
openssl rsa -in ~/.energy-monitor/keys/private.pem -pubout -out ~/.energy-monitor/keys/public.pem
```

On Windows (PowerShell or Git Bash), with the same hidden `.energy-monitor` folder:

```
New-Item -ItemType Directory -Force "$HOME\.energy-monitor\keys"
openssl genpkey -algorithm RSA -out "$HOME\.energy-monitor\keys\private.pem" -pkeyopt
rsa_keygen_bits:2048
openssl rsa -in "$HOME\.energy-monitor\keys\private.pem" -pubout -out "$HOME\.energy-
monitor\keys\public.pem"
```

Set the absolute paths (Java does not expand `~`) in each service's `.env`: `SECURITY_JWT_PRIVATE_KEY_PATH` and `SECURITY_JWT_PUBLIC_KEY_PATH` in the security service, and only `SECURITY_JWT_PUBLIC_KEY_PATH` in the gateway and the other services. All must use the same pair.

```
SECURITY_JWT_PRIVATE_KEY_PATH=C:\Users\your-user\.energy-monitor\keys\private.pem
SECURITY_JWT_PUBLIC_KEY_PATH=C:\Users\your-user\.energy-monitor\keys\public.pem
```

> *The `.pem` keys are secrets: never commit them or paste them in a chat.*

9.6 Generate the recovery pepper

The block in section 9.2 already generates it in `infra/.env`. To create one by hand:

```
openssl rand -hex 32
```

Copy the same value into `SECURITY_RESET_PEPPER` in the security service's `.env`. Without a pepper the security service refuses to start, just as without the RSA keys. Changing it invalidates recovery codes already issued.

9.7 Decide on the recovery email

Choose one of two options: use Mailpit locally (recommended for development) or use a real Gmail account with a 16-character "app password" generated in the Google account (Google, n.d.). Email is sent by the notification service: the `MAIL_*` values go in `infra/.env` and in that service's `.env`. For Mailpit use `MAIL_HOST=localhost`, `MAIL_PORT=587`, `MAIL_USERNAME=dev`, `MAIL_PASSWORD=dev`; messages are read at `http://localhost:8025` (section 16).

9.8 Build and start

In development mode, with MySQL, RabbitMQ and the broker up (section 9.3), start each service in its own terminal, in this order:

```
cd backend/en-monitor-security-service    && ./mvnw spring-boot:run   # :8082
cd backend/en-monitor-device-service      && ./mvnw spring-boot:run   # :8084
cd backend/en-monitor-measurement-service && ./mvnw spring-boot:run   # :8085
cd backend/en-monitor-home-service        && ./mvnw spring-boot:run   # :8086
```

```
cd backend/en-monitor-notification-service && ./mvnw spring-boot:run  # :8087
cd backend/en-monitor-alert-service       && ./mvnw spring-boot:run   # :8088
cd backend/en-monitor-recommendation-service && ./mvnw spring-boot:run # :8089
cd backend/en-monitor-api-gateway         && ./mvnw spring-boot:run   # :8080
# Windows: .\mvnw.cmd spring-boot:run in each folder
```

For the tests, in each repository: `./mvnw clean test` (on Windows, `.\mvnw.cmd clean test`). Each repository includes the Maven Wrapper, so there is no need to install Maven separately. The API runs at `http://localhost:8080` (the gateway); each service's OpenAPI documentation at `/v3/api-docs` and Swagger UI at `/swagger-ui.html` on its own port (`http://localhost:8082` and `:8084` to `:8089`) (Spring, n.d.). The health status is at `/actuator/health` of each service and of the gateway.

> *The tests are not hermetic: the persistence tests run against the real database (Liquibase creates the schema). That is why MySQL must be healthy before* `mvnw test`*. The gateway does not need a database.*

10. Web frontend installation

Repository: `Energy-Monitor-application` (React 19 + Vite 8). Dev server at `http://localhost:5173` (Vite, n.d.).

10.1 Dependencies

With Node 22 and npm in the PATH, from the repository folder:

```
npm install
```

10.2 Configure the development gateway

The web always calls `/api` on its own origin and the Vite server forwards it to the gateway at `http://localhost:8080`; that way the HttpOnly session cookies work without CORS. A `.env.local` is only needed if you change a default value:

```
cp .env.example .env.local
# Windows PowerShell:
Copy-Item .env.example .env.local
```

```
# VITE_DEV_API_TARGET=http://localhost:8080
VITE_GOOGLE_CLIENT_ID=
```

`VITE_GOOGLE_CLIENT_ID` must be the same client ID as the security service's `GOOGLE_CLIENT_ID`. If left empty, the Google button is not shown.

10.3 Start in development

```
npm run dev
```

Open `http://localhost:5173`. To build for production: `npm run build`; to check the code: `npm run lint`; for the tests: `npm test` (Vite, n.d.).

10.4 CORS permission in the gateway

CORS is answered by the API gateway. Its `.env` must list the Vite origin as an allowed origin:

```
SECURITY_CORS_ALLOWED_ORIGINS=http://localhost:5173
```

> *In the full stack the origin is `http://localhost` and it is set in `infra/.env`. There the web and the API share an origin through the frontend's nginx, which forwards `/api/` to the gateway.*

11. Mobile application installation

Repository: `Energy-Monitor-Mobile` (Expo SDK 57, React Native 0.86.3, TypeScript, Android only). The application is integrated with the API, so the gateway and the services must be running to sign in and query data (Expo, n.d.).

11.1 Dependencies

```
npm install
```

11.2 Configure the API URL

Copy the example and set the gateway address, without `/api/v1`:

```
cp .env.example .env
# Windows PowerShell:
Copy-Item .env.example .env
```

```
EXPO_PUBLIC_API_BASE_URL=http://10.0.2.2:8080
EXPO_PUBLIC_EAS_PROJECT_ID=<project id in expo.dev>
```

The value depends on where the application runs:

Where the app runs	Value of EXPO_PUBLIC_API_BASE_URL
Android emulator	`http://10.0.2.2:8080` (`localhost` would be the emulator itself)
Physical phone	`http://<LAN IP of the PC>:8080` in development mode, for example `http://192.168.1.50:8080`, or `http://<IP of the PC>` with the full stack
Browser (`npm run web`)	`http://localhost:8080`

Expo embeds the `EXPO_PUBLIC_*` variables at build time: after changing the value, restart `npx expo start`. CORS does not apply to React Native, so the app does not need to be added to `SECURITY_CORS_ALLOWED_ORIGINS`. The Expo push token (`EXPO_ACCESS_TOKEN`) never goes in this file: it belongs to the notification service. To use Google sign-in in the app, add `EXPO_PUBLIC_GOOGLE_WEB_CLIENT_ID` with the **web** client ID and create, in the same Google project, an Android client with the `com.energymonitor.mobile` package and the SHA-1 of the key that signs the APK.

11.3 Start Expo

```
npm start
# or directly:
npx expo start
```

The Expo CLI shows a QR code and interactive menus. Choose the platform:

- Physical Android or emulator: `npm run android` (or press `a`).

- Browser, only as a development aid: `npm run web` (or press `w`).

11.4 Test on a physical device

Install Expo Go from Google Play, make sure the computer and the phone are on the same network, and scan the QR from Expo Go. It is the recommended route on Windows because it does not require Android Studio (Expo, n.d.). For the phone to reach the gateway, allow port 8080 (development mode) or 80 (full stack) in the computer's firewall.

11.5 Compatibility notes

- Node 22 is the reference environment for the Expo SDK 57 tooling.
- The mobile application is not available for iPhone: this manual does not cover iOS or Xcode.
- Linking the module from the app uses native Bluetooth (`react-native-ble-plx`): grant the Bluetooth permission when the app asks. Push notifications need `EXPO_PUBLIC_EAS_PROJECT_ID` in the app and `EXPO_ACCESS_TOKEN` on the server.
- The app keeps the session in the system secure store and has Android backups disabled: after reinstalling it, sign in again.
- If the QR does not open, check the firewall or start with `npx expo start --tunnel`.

12. MQTT broker

The broker is Eclipse Mosquitto with the `mosquitto-go-auth` plugin, and its configuration lives in `energy-monitor-infra/mosquitto/`. It does not store users: for each connection and each publication it asks the device service (`POST /internal/mqtt/auth/user` and `/acl`, cached for 5 minutes), and each device can only publish to its own topics. It is started with the infrastructure compose (section 9.3), in the full stack or in development mode (Eclipse Foundation, n.d.).

Clients the broker accepts: the device service (`em-device-service`, reads telemetry and status and is the only one that publishes commands), the measurement service (`em-measurement-service`, only reads telemetry to store the readings) and each ESP32 device (`device_code` and `api_key`). No other service connects to the broker: the alert service receives each reading through RabbitMQ (`measurement.recorded`).

Steps:

1. Set in `infra/.env` `DEVICE_MQTT_PUBLIC_HOST` (the LAN IP of the PC; on Linux, `hostname -I`), `DEVICE_MQTT_PUBLIC_PORT=8883`, `MQTT_DEVICE_SERVICE_PASS` and `MQTT_MEASUREMENT_SERVICE_PASS` (section 9.2); in development mode, the same values in the `.env` files of the device and measurement services (section 9.4).
2. Generate the development TLS certificate with `./prepare.sh` from `infra/` (it calls `mosquitto/generate-dev-certs.sh`). The script creates a development certificate authority and the broker certificate in `infra/mosquitto/certs/`, valid for the machine's IPs and always for `DEVICE_MQTT_PUBLIC_HOST`, and writes `backend/energy-monitor-backend/firmware/esp32-pzem-monitor/dev_ca.h` so the firmware trusts that authority. If the machine's IP changes, update `DEVICE_MQTT_PUBLIC_HOST` and run it again: the authority is reused and the devices do not need to be reprogrammed.
3. In development mode, the broker calls the device service running on the machine (through `host-gateway`). On Linux with ufw, allow that call once, from the Docker networks only:

```
sudo ufw allow from 172.16.0.0/12 to any port 8084 proto tcp comment 'mosquitto -> device'
```

4. With the device service running, check the broker from `infra/`:

```
docker compose logs -f mosquitto
```

The services show up connected as `em-device-service` and `em-measurement-service`. Devices use port 8883 with TLS; 1883 is internal, for the services only, and Docker publishes it on the loopback interface only. To go back to the full stack, `docker compose up -d` recreates the broker without `host-gateway`.

> *If the container exits with a permission error on `server.key`, the container reads the file as user 1000: run `sudo chown 1000 infra/mosquitto/certs/server.key`.*

13. Measuring device firmware

The firmware (version 1.3.8) is in `backend/energy-monitor-backend/firmware/esp32-pzem-monitor/` (`energy-monitor-backend` repository) and is built with arduino-cli or Arduino IDE 2 (Arduino, n.d.). There is no over-the-air (OTA) update: the firmware is always uploaded over USB.

13.1 Wiring

PZEM-004T V3	ESP32	Note
TX	GPIO16 (RX2)	TX and RX are crossed
RX	GPIO17 (TX2)	
VCC	3V3	If it does not answer, use 5 V from VIN
GND	GND	Common with the PZEM

Communication is Modbus RTU at 9600 baud, 8N1, with address `0xF8`. The HLK-PM03 power supply powers the board from the same circuit being measured.

> **Safety warning:** the PZEM-004T is not galvanically isolated. Do not touch the side connected to the mains and do not handle the mains wiring with the prototype powered.

13.2 Tools and libraries

```
arduino-cli core update-index
arduino-cli core install esp32:esp32
arduino-cli lib install PZEM004Tv30 PubSubClient ArduinoJson WiFiManager NimBLE-Arduino
```

Library	Author	Tested version
PZEM004Tv30	Jakub Mandula	1.2.1
PubSubClient	Nick O'Leary	2.8
ArduinoJson	Benoit Blanchon	7.x
WiFiManager	tzapu	2.0.17
NimBLE-Arduino	h2zero	2.5.1

Do not confuse `PZEM004Tv30` with the old `PZEM-004T` library (v1/v2).

13.3 Preparation

```
cd backend/energy-monitor-backend/firmware/esp32-pzem-monitor
cp secrets.h.example secrets.h
```

Set at least `PORTAL_PASSWORD` (8 characters or more) in `secrets.h`: it protects the `EnergyMonitor-Setup` access point. The `dev_ca.h` file is generated by `prepare.sh` in `energy-monitor-infra` (section 12). Both files are ignored by Git.

13.4 Build and upload over USB

The partition scheme must be **Minimal SPIFFS (1.9 MB APP with OTA)**, because with Bluetooth the binary (~1.4 MB) does not fit in the default scheme.

```
arduino-cli compile --fqbn esp32:esp32:esp32:PartitionScheme=min_spiffs .
arduino-cli upload  --fqbn esp32:esp32:esp32:PartitionScheme=min_spiffs -p /dev/ttyUSB0 .
arduino-cli monitor -p /dev/ttyUSB0 -c baudrate=115200
```

In Arduino IDE: board **ESP32 Dev Module** and *Partition Scheme* **Minimal SPIFFS**. Close any other serial monitor before uploading, or the port will be busy. On Linux, the user needs access to the port (`sudo usermod -aG dialout $USER` and log in again).

13.5 Setup mode

A new device, with no configuration, enters setup mode on its own. To activate it on one that is already configured, hold the **BOOT** button (GPIO0) for 3 seconds. With no known network for 5 minutes, the device offers Bluetooth configuration by itself.

14. Link the first device

The device is registered from the web application or from the mobile app; nothing needs to be inserted manually into the database. Requirements: database, RabbitMQ, broker, gateway and services running, and the web open in desktop Chrome or Edge, or Chrome on Android, over HTTPS or `localhost` (in the mobile app, Bluetooth on and its permission granted).

1. In the web or the app: household → **Devices** → **Link device** (owner only).
2. Hold **BOOT** on the ESP32 for 3 seconds until the LED blinks (a device that was never linked is already in setup mode after uploading).
3. **Find device**: the browser shows `EnergyMonitor-<code>`; select it.
4. Choose the appliance, the name and the room.
5. Choose the household Wi-Fi from the list (they are the networks the ESP32 sees) and type its password.
6. The web or the app registers the device in the device service, which generates its `api_key`, and sends the device over Bluetooth the network, the password, the server (`DEVICE_MQTT_PUBLIC_HOST`), the identifier and the key.
7. Once connected to the broker, the device leaves setup mode on its own.

Alternative without Bluetooth (Safari, Firefox or other browsers): join the `EnergyMonitor-Setup` Wi-Fi network with the `PORTAL_PASSWORD` password, open `http://192.168.4.1` → **Configure WiFi** and fill in the network, the MQTT server (the PC's IP or the broker's domain), port 8883, the identifier, the code and the device key. This route requires the device to already exist in the device service with its key.

> *On Linux, Chrome ships with Web Bluetooth disabled: enable* `chrome://flags/#enable-experimental-web-platform-features` *and restart the browser. Web Bluetooth only works on* `https://` *or* `http://localhost`.

Expected output in the serial monitor:

```
=== EnergyMonitor ESP32 + PZEM-004T V3 fw=1.3.8 ===
[PZEM] UART2 ready at 9600 baud, addr=0xf8
[WiFi] connected to 'MyHome', IP=192.168.1.37
[MQTT] connected as esp32-DEV0000042
[MQTT] telemetry published
```

15. Production

This section summarizes deployment on a server with Docker Compose from `energy-monitor-infra` (`docker-compose.prod.yml` and `.env.prod`): MySQL, RabbitMQ, the broker, the API gateway, the seven services and the frontend. The full procedure, with verification, rollback and continuous deployment, is in the implementation and deployment manual. The `mosquitto.conf` is the same file as in development (Let's Encrypt, n.d.).

1. **Server and domain.** A server with Docker, Nginx and Certbot, and DNS records for the web domain, `www` and the broker (for example `mqtt.example.com`) pointing to it.
2. **Code.** The repositories cloned side by side in `/opt/energy-monitor` (`infra`, the gateway and the seven services under `backend/`, the retired backend only for the firmware and `frontend/energy-monitor-web`), as in section 9.1, on the `main` branch.
3. **Broker certificate.** Obtain a Let's Encrypt certificate for the broker's domain (`certbot certonly --nginx -d mqtt.example.com`); the `deploy/renew-mqtt-cert.sh` hook copies `fullchain.pem` as `server.crt` and `privkey.pem` as `server.key` into the `MQTT_CERT_DIR` folder (readable by user 1000) and restarts `mosquitto` on every renewal. The firmware does not change: it trusts the Let's Encrypt roots.
4. **Configuration.** `cp .env.prod.example .env.prod` in `infra/` and fill it in: MySQL passwords, pepper, `RABBITMQ_PASSWORD`, `INTERNAL_API_TOKEN`, `MQTT_DEVICE_SERVICE_PASS`, `MQTT_MEASUREMENT_SERVICE_PASS`, the JWT key pair folder, `DEVICE_MQTT_PUBLIC_HOST` with the broker's domain, `https://<domain>` as origin and public URL, email, Google, VAPID and Expo. The JWT private key must be readable by `BACKEND_UID` (10001 by default), the user the security service runs as; `public.pem`, by everyone. `chmod 600 .env.prod`.
5. **Start.**

```
cd infra
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d --build
```

6. **Images.** Each Java service's `Dockerfile` builds with Maven and runs with a Java 21 JRE as an unprivileged user (UID 10001). No service publishes ports: only the web on `127.0.0.1:8081` (by default), MQTT with TLS on 8883 and the RabbitMQ console on `127.0.0.1:15672` are published.
7. **Firewall.** Open ports 22, 80, 443 (HTTPS proxy) and 8883 (MQTT with TLS). Keep 1883, 3306, 8081 and 15672 closed.
8. **HTTPS reverse proxy** (Nginx with `deploy/nginx-energy-monitor.conf`) in front of `127.0.0.1:8081` for the web and the API. The frontend's nginx only passes `/api/` to the gateway, and the gateway rejects `/internal/**`. The web must be served over HTTPS, because Web Bluetooth and push require it.
9. **Real client address.** The Nginx site replaces `X-Forwarded-For` with the real IP, and the gateway and the security service only accept it if the connection comes from the Docker network; that way the attempt limit is not shared among all users.
10. **First administrator and backups.** Register the account in the web and run `./make-admin.sh --prod email@example.com`; schedule `deploy/backup-mysql.sh` in root's cron.

VAPID keys are generated once per environment (changing them invalidates every push subscription); the commands are in the `energy-monitor-infra` README. Several instances of the device or measurement services need different `MQTT_CLIENT_ID` values. A server that still runs `energy-monitor-backend` is migrated with the "Retiro de energy-monitor-backend" procedure of the `energy-monitor-infra` README (backup, purge the alert queues, check `backend_outbox`, `deploy.sh infra` and `deploy.sh gateway`, delete the `energy-monitor-`

`backend.*` queues). When a service that consumes new events is deployed, it must go **before** the one that produces them: an outbox stops at a message no queue takes.

16. Local email with Mailpit (optional)

Mailpit is a development SMTP server: it accepts any message and shows them in a web inbox without sending anything over the real internet. It is started with Docker (Mailpit, n.d.):

```
docker run -d --name mailpit -p 587:1025 -p 8025:8025 \
  -e MP_SMTP_AUTH_ACCEPT_ANY=1 -e MP_SMTP_AUTH_ALLOW_INSECURE=1 axllent/mailpit
```

Use port 587 on the machine (instead of Mailpit's default 1025) so that when switching between Mailpit and Gmail only `MAIL_HOST` and the credentials change. In the notification service's `.env` (development mode):

```
MAIL_HOST=localhost
MAIL_PORT=587
MAIL_USERNAME=dev
MAIL_PASSWORD=dev
MAIL_FROM=Energy Monitor <no-reply@localhost>
```

Development inbox: `http://localhost:8025`. When you use Gmail, the recovery code arrives in the real mailbox associated with the account with the "app password".

17. Installation verification

Run this checklist before considering the installation finished. Each command must produce the indicated result:

Component	Check	Expected result
Docker	`docker compose ps` in `infra/`	`energy-monitor-mysql` and `energy-monitor-rabbitmq` in `healthy` state; in the full stack, also gateway, security, device, measurement, home, notification, alert and recommendation `(healthy)`.
Web and API	`curl -s -o /dev/null -w '%{http_code}\n' http://localhost/` (full stack)	`200`.
Gateway	`GET http://localhost/api/v1/homes` (full stack) or `http://localhost:8080/api/v1/homes` (development mode), without a session	`401`: answered by the gateway.
Routes per service	`POST /api/v1/auth/login` with `{}`; `GET /api/v1/appliance-types`, `/consumption-levels`, `/home-types`, `/alerts?homeId=x`, `/recommendations?homeId=x`, `/notifications/preferences`; `GET /api/v1/other-route`	`400` (reached the security service); `401` on the others (routes of each service); `404` on the route nobody serves.
Services	`http://localhost:8080/actuator/health`, `:8082` and `:8084` to `:8089` (development mode)	Each answers `UP`.
Swagger	`http://localhost:8082/swagger-ui.html` (and `:8084` to `:8089`) in development mode	Swagger UI lists each service's documentation.
OpenAPI	`GET http://localhost:8086/v3/api-docs` (or another service port)	JSON with the OpenAPI specification.
Broker	`docker compose logs mosquitto | grep "New client connected"` in `infra/`	`em-device-service` and `em-measurement-service` connected; port 8883 published.

Component	Check	Expected result
RabbitMQ	`http://localhost:15672` (`RABBITMQ_USERNAME` / `RABBITMQ_PASSWORD`)	The console opens and the `.dlq` queues are empty.
Device	ESP32 serial monitor	`[MQTT] connected as esp32-<identifier>` and `[MQTT] telemetry published`.
First measurement	Household panel in the web	The device's first measurement appears in the web in about one minute.
Web (development)	`npm run dev` in Energy-Monitor-application	Vite at `http://localhost:5173` shows the sign-in screen.
Mobile	`npm start` in Energy-Monitor-Mobile	Expo shows the QR and, when opened with Expo Go, the app lets you sign in against the API.
Email	`MAIL_HOST=localhost` + Mailpit on :587	Recovery codes appear at `http://localhost:8025`.
Administrator	`./make-admin.sh email@example.com` in `infra/` and sign in again on the web	**Administration** appears in the side menu and the `/admin/users` screen opens.

For an end-to-end check: register a user from the web, verify their email (Mailpit or Gmail), log in, create a household, link the device (section 14) and confirm that its first measurement appears (Cortés Parra & García Artunduaga, 2026). With the full stack up, the `infra/e2e/` scripts (`device-flow.sh`, `measurement-flow.sh`, `activity-flow.sh`, `alert-flow.sh`, `recommendation-flow.sh` and `account-deletion-flow.sh`) automatically walk through what crosses the services (gateway routes, linking, broker, readings, alerts, recommendations, activity and account deletion) and finish with the number of checks passed; they use `jq` and `openssl`.

18. Installation command summary

Step	Infrastructure (energy-monitor-infra)	Java services (gateway and the seven services)	Web (Energy-Monitor-application)	Mobile (Energy-Monitor-Mobile)
Variables	`cp .env.example .env` and secrets (9.2); first administrator with `./make-admin.sh`	`cp .env.example .env` and `application-local.yaml` (9.4, development mode)	`cp .env.example .env.local` (optional)	`cp .env.example .env`
Deps.	—	(Maven wrapper included)	`npm install`	`npm install`
Keys/certificates	`./prepare.sh`	Key paths in `.env` (9.5)	—	—
Infrastructure	Full stack: `docker compose up -d --build`; development: `docker compose -f docker-compose.yml -f docker-compose.dev.yml up -d mysql rabbitmq mosquitto`	—	—	—
Tests	`./e2e/*.sh` (full stack)	`./mvnw test` (`.\mvnw.cmd test` on Windows)	`npm test`	`npm run lint`
Run	—	`./mvnw spring-boot:run` in each service (9.8)	`npm run dev`	`npm start`
Typical URL	`http://localhost` (full stack)	`http://localhost:8080` (gateway)	`http://localhost:5173`	QR / localhost:8081 (Expo)

Broker: `./prepare.sh` in `infra/` generates its certificates and the compose starts it. Firmware (`backend/energy-monitor-backend/firmware/esp32-pzem-monitor`): `arduino-cli compile` and `arduino-cli upload` (section 13).

19. Common troubleshooting

Symptom	Probable cause	Corrective action
`required variable X is missing a value` when starting the stack	`X` is empty in `infra/.env` or its line is missing.	Fill it in (section 9.2) and repeat `docker compose config -q` until it names none.
The security service does not start and names a key path or the pepper	`SECURITY_JWT_PRIVATE_KEY_PATH`, `SECURITY_JWT_PUBLIC_KEY_PATH` or `SECURITY_RESET_PEPPER` are missing.	Define them in its `.env` (absolute paths) and retry.
`Access denied for user 'energy_app'` in a service or when running tests	Its `SPRING_DATASOURCE_PASSWORD` or its `application-local.yaml` do not match `MYSQL_PASSWORD` in `infra/.env`.	Copy the `MYSQL_PASSWORD` value (section 9.4).
A service does not start because of a `.env` value	A value with quotes or `$` (the file is read as `.properties`).	Write values without quotes and without `$`.
A service stays `unhealthy` with RabbitMQ up	`RABBITMQ_USERNAME` / `RABBITMQ_PASSWORD` do not match the user RabbitMQ created the first time its volume started.	Change the password inside RabbitMQ (`rabbitmqctl change_password`, see the infra README) or delete the `rabbitmq-data` volume (pending messages are lost).
The API answers `503`	The gateway cannot reach the service of that route.	Check `docker compose ps` and `docker compose logs security device measurement home notification alert recommendation` (or the service's terminal in development mode).
The browser blocks calls from the web	`SECURITY_CORS_ALLOWED_ORIGINS` with the Vite origin is missing in the gateway.	Add it in the gateway's `.env` (section 10.4).
The frontend keeps getting `401` on `/api/v1/homes`	It is expected without a session.	Sign in (registration or Google).
Verification or recovery codes do not arrive	Email misconfigured in the notification service; the error is recorded in the `notification` table.	Check `MAIL_*` of the notification service, the `notification-service.notification.mail` queue and its `.dlq`, use Mailpit and read the inbox at `:8025` (section 16).
An event does not arrive (alerts, activity, home deletion)	The producer's outbox is stopped because the consumer's queue does not exist.	Read `last_error` of the `*_outbox` table, deploy the consumer and wait for the relay to move.
Readings arrive but no alerts are created	The alert service is stopped or has `ALERT_CONSUMERS_ENABLED=false`.	Start `alert`, remove the `false` from `.env` and check the `alert-service.measurement.recorded` queue.
Mailpit/Docker does not start	Docker Desktop stopped or port in use.	Start Docker Desktop; change the port with `-p 1587:1025` if 587 is in use.
The device does not connect (`[MQTT] connection failed, rc=-2` or `X509 - Certificate verification failed`)	`DEVICE_MQTT_PUBLIC_HOST` points to `localhost` or a wrong IP, the certificate is not valid for the current IP, or the network blocks it.	Use the PC's LAN IP, run `./prepare.sh` again, `docker compose up -d --force-recreate mosquitto` and check the firewall.
The device shows `rc=4` or `rc=5`	The code or key do not match the device service's, or that service is stopped.	Start the device service and link the device again.

Symptom	Probable cause	Corrective action
A service is rejected by the broker (`rc=5`)	`MQTT_MEASUREMENT_SERVICE_USER/PASS` or `MQTT_DEVICE_SERVICE_USER/PASS` are not the same for the client that connects and in the device service.	Make the values equal in `infra/.env` and in the `.env` files of both services.
The broker logs `context deadline exceeded` calling `device:8084` (development mode)	The device service on the machine is stopped or the ufw rule is missing.	Start the device service and apply the rule from section 12 (with port 8084).
The services take about 2 minutes on the first start	Liquibase runs its changesets on the new database.	It is normal; wait for `(healthy)` in `docker compose ps`.
The web does not find the device over Bluetooth	The device is not in setup mode, the browser is not compatible or the page is not secure.	Hold BOOT for 3 s until the LED blinks, use Chrome or Edge and open the web on `localhost` or over HTTPS; on Linux enable the flag from section 14.
`wifi_failed:auth` while linking	Wrong Wi-Fi password.	Link again with the correct password.
Browser push does not arrive	The VAPID keys are missing or the user did not grant notification permission; the browser requires HTTPS or `localhost`.	Define `NOTIFICATION_VAPID_PUBLIC_KEY` and `NOTIFICATION_VAPID_PRIVATE_KEY`, restart the notification service and enable notifications in the browser.
Google answers `origin_mismatch` or the button does not appear	The origin is missing from *Authorized JavaScript origins* of the OAuth client, or `GOOGLE_CLIENT_ID` / `VITE_GOOGLE_CLIENT_ID` are empty.	Register `http://localhost` and `http://localhost:5173` in Google Cloud Console, set the variables and rebuild the frontend or restart Vite.
Firmware upload fails: port busy	Another program is using the serial port.	Close the Arduino IDE serial monitor or any other program using `/dev/ttyUSB0`.
"Sketch too big" when compiling	The partition scheme is not Minimal SPIFFS.	Compile with `PartitionScheme=min_spiffs` (section 13.4).
"~" is not expanded in the key path	Java does not expand the symbol in the key paths.	Use the full absolute path (`C:\Users\your-user\.energy-monitor\keys\`).
The mobile app does not reach the API	`EXPO_PUBLIC_API_BASE_URL` points to `localhost`, which on the emulator or the phone is the device itself.	Use `10.0.2.2` on the emulator or the PC's IP on the phone and restart Expo (section 11.2).
Expo does not connect with Expo Go	Different network, firewall, or invalid QR.	Use `npx expo start --tunnel`; check corporate network scripts.
`npm install` fails on mobile/web	Node version different from the reference software.	Update to Node 22 LTS (min. 20.19) and reinstall.

20. References (APA format, 7th edition)

In-text citations and the reference list follow the style of the seventh edition of the APA publication manual (American Psychological Association [APA], 2020).

American Psychological Association. (2020). Publication manual of the American Psychological Association (7th ed.). https://www.apa.org/

Arduino. (n.d.). Arduino CLI. https://arduino.github.io/arduino-cli/

Cortés Parra, J. E., & García Artunduaga, M. A. (2026). *EnergyMonitor software requirements specification* (version 2.0). SENA – CIES Huila Regional.

Docker Inc. (n.d.). Docker documentation. https://docs.docker.com/

Eclipse Foundation. (n.d.). Eclipse Mosquitto. https://mosquitto.org/

Expo. (n.d.). Expo documentation. https://docs.expo.dev/

Git. (n.d.). Git for Windows. https://git-scm.com/download/win

Google. (n.d.). How to create an app password. https://myaccount.google.com/apppasswords

Let's Encrypt. (n.d.). Documentation. https://letsencrypt.org/docs/

Mailpit. (n.d.). Mailpit: Web and API based SMTP testing. https://mailpit.axllent.org/

RabbitMQ. (n.d.). RabbitMQ documentation. https://www.rabbitmq.com/docs

MySQL. (2026). MySQL 8.4 reference manual. https://dev.mysql.com/doc/refman/8.4/en/

Node.js Foundation. (n.d.). Node.js. https://nodejs.org/

OpenSSL. (n.d.). OpenSSL cryptography and SSL/TLS toolkit. https://www.openssl.org/

Oracle. (n.d.). Java SE downloads. https://www.oracle.com/java/technologies/downloads/

Spring. (n.d.). Spring Boot reference documentation. https://docs.spring.io/spring-boot/index.html

Vite. (n.d.). Vite guide. https://vite.dev/guide/